



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

Chapter 2

Java Fundamentals



Chapter 2

- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- Inputs and outputs
- Array
- Exception mechanisms
- Java Virtual Machine and garbage collection



Identifier

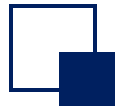
❑ **Identifier:** A valid character series used to identify class names, variable names, method names, object names, array names, and file names.

❑ **Specification of identifiers:**

The identifier can be a combination of letters, numbers, underscores, and dollar sign \$, and the length is unlimited.

The identifier must start with an alphabet, an underscore, and dollar sign \$, and **cannot start with a number.**

Java identifiers are case-sensitive.



Identifier

NO.1

Capitalize the first letter of each word of the **class name**, such as: **HelloWorldClass**.

NO.4

Constant names are all capitalized, separated by underscores. As: **MONTH_OF_YEAR**.

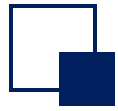
Java naming conventions

NO.2

Identifiers such as **variable names, object names, method names, and package names** are all in lowercase letters. If the identifier consists of more than one word, its first letter is lowercase, and all words after it should capitalize the first letter, e.g. getAge.

NO.3

It cannot have the same name as a keyword or a special value, such as: **false, for, while**



Keywords

❑ Data Type Keywords

- int, double, boolean, char, float, long, short, byte

❑ Control Flow Keywords

- if, else, while, for, switch, case, break, continue

❑ Class and Method Keywords

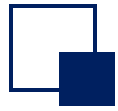
- class, public, private, protected, static, void, return

❑ Object Creation Keywords

- new, this, super, extends, implements

❑ Exception Handling Keywords

- try, catch, finally, throw, throws



separator

semicolon ;

The end of the
identification
statement cannot be
omitted.

For example:

```
{x=12;y=4;}
```

braces { }

Indicate the scope of
the class, the scope of
the method body, the
scope of the compound
statement, and the
initialization of array
members. as:

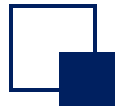
```
if condition {...}  
else {...}
```

bracket (), []

() flags used for cast
castings or used as
function headers;[] is
used for the definition
of arrays and
references to elements。

For example:

```
int[] score = new int[30];
```



Comments

Line Comments

//

```
// Add below  
int a = 1 + 2;
```

Block Comments

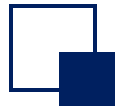
/* */

```
/* Program Name:  
   Function:  
   Version:  
   Writing time: */
```

Document Comments

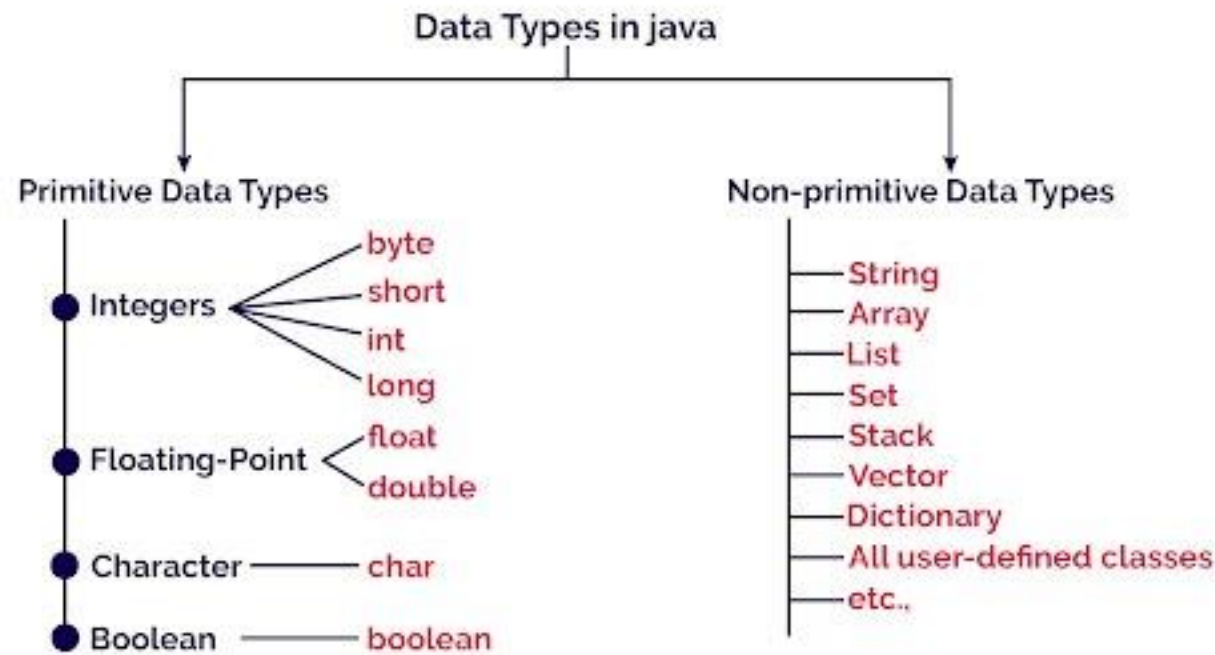
/** */

```
/**  
    Document  
    annotation content  
    */  
The content of the  
document comments  
can be automatically  
generated by using the  
javadoc command.
```



Java basic data types

- ❑ Java divides variables into two categories based on data types: primitive data types and non-primitive data types
- ❑ There are eight primitive data types: byte, short, int, long, float, double, char, and boolean



Integer Type: byte、 short、 int、 long

- ❑ Each integer type in Java has a fixed range of tables and field lengths, which are not affected by the specific operating system, so as to ensure the portability of Java programs
- ❑ Java's shaping constants default to int, declaring long constants must be accompanied by 'l' or 'L' at the end of the number

type	storage space	The range of numbers
byte	1byte=8bit	-128 ~ 127
short	2byte	$-2^{15} \sim 2^{15}-1$
int	4byte	$-2^{31} \sim 2^{31}-1$ (About 2 billion)
long	8byte	$-2^{63} \sim 2^{63}-1$

Floating-point type: float、 double

- ❑ The floating-point type of Java adopts the IEEE floating-point format, which has a fixed range of tables and field lengths, and is not affected by the specific operating system.
- ❑ Java's floating-point constants default to double, and float-type constants must be declared with 'f' or 'F' at the end of the number.

type	Takes up storage space	The range of table numbers
Single-precision float	4byte	-3.403E38 ~ 3.403E38
Double-precision double	8byte	-1.798E308 ~ 1.798E308

Floating-point type: float、double

❑ The floating-point type constructor:

```
float f = 3.14f  
float f = new Float(3.14)
```

```
float f = 3.14
```



❑ Type Promotion :

```
public class Main {  
    public static void main(String[] args) {  
        int n = 5;  
        double d = (1.2 + 24.0)/n;  
        System.out.println(d); //5.04  
    }  
}
```

```
double d = 1.2 + 24 / 5;
```

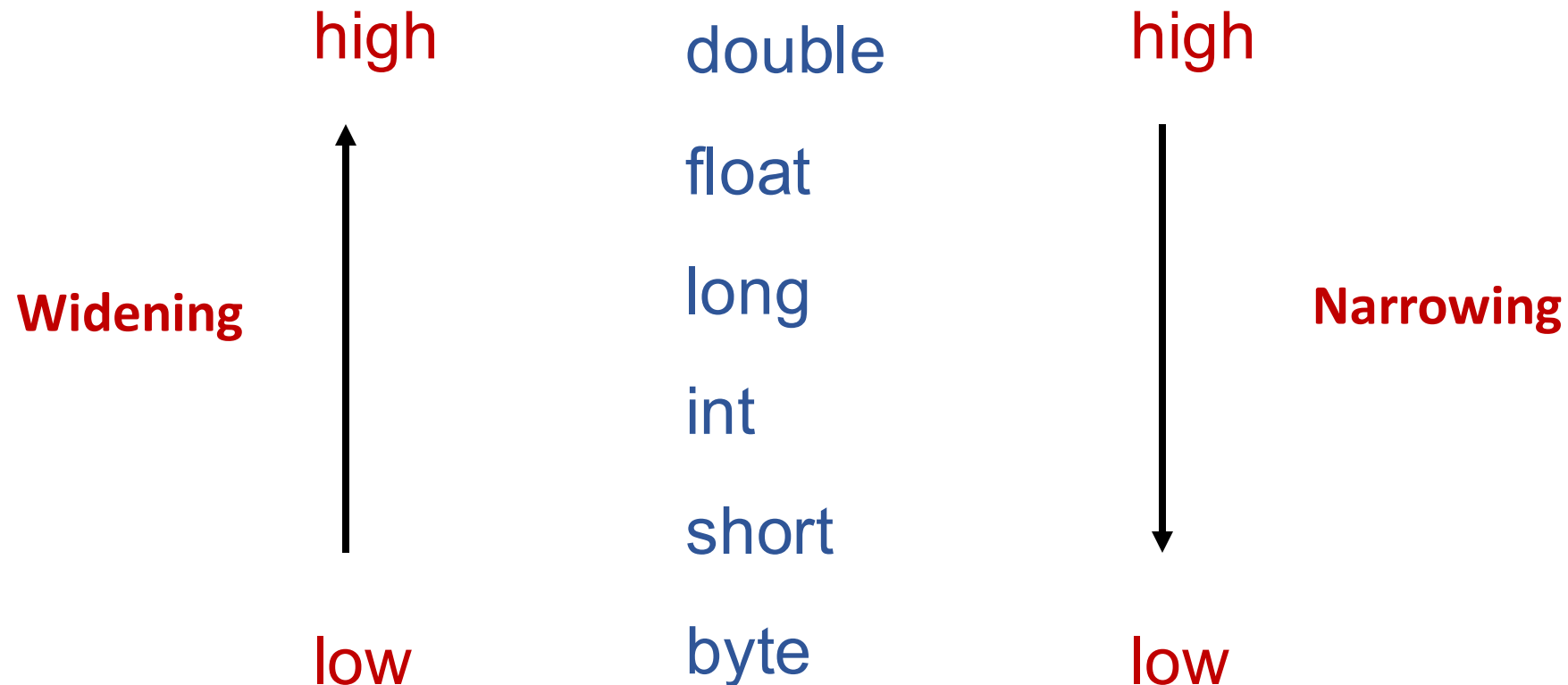
```
// 5.2
```

?



Conversion between different types of data

- A conversion of a data type occurs when the value of one data type is assigned to a variable of another data type.
- In the integer type and the floating-point number, you can arrange the data types in order of precision from "high" to "low" as follows:



Type conversion rules

- ❑ When a low-level value is assigned to a high-level variable, the system automatically completes the conversion of the data type.

Example: `float x=200;` Convert the int value of type 200 to the value of float type 200.0

- ❑ When a high-level value is assigned to a low-level variable, a cast conversion must be performed.
 - Forced syntax format: (target type name) the value to be converted; The value can be an expression

Example: `int i;`

`i = (int) 26L;` Convert the value of type long 26 to the value of type int 26.

- **When you perform a cast conversion, you may lose data precision.**

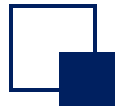
String numbers are converted to numeric values

❑ Example of converting to an integer:

```
int x1 = Integer.parseInt("260"); //String direct conversion, x1=260  
int x2 = Integer.parseInt(txt1.getText()); //Convert the text in the text box txt1 to int and assign it  
to x2
```

❑ Example of converting to floating-point:

```
float y = Float.parseFloat("23.5");  
double z = Double.parseDouble("45.6");
```



Character Type: char

- ❑ The Java language specification states that the char type of Java is a UTF-16 code unit, which is 16 bits (2 bytes)

- ❑ There are three types of character-based variables:

- A character variable is a single character enclosed in single quotation marks. For example:

```
char c1 = 'a'; char c2 = '中' ; char c3 = '9';
```

- Java also allows the use of the escape character '\\' to convert subsequent characters into special character constants. For example:

```
char c3 = '\n';
```

Indicates a line break

- Use the Unicode value directly to represent the character constant: '\uXXXX'. where XXXX represents a hexadecimal integer. For example, \u000a means \n.

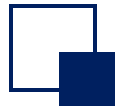
- ❑ The char type can be computed because it corresponds to Unicode code.



Boolean type: boolean

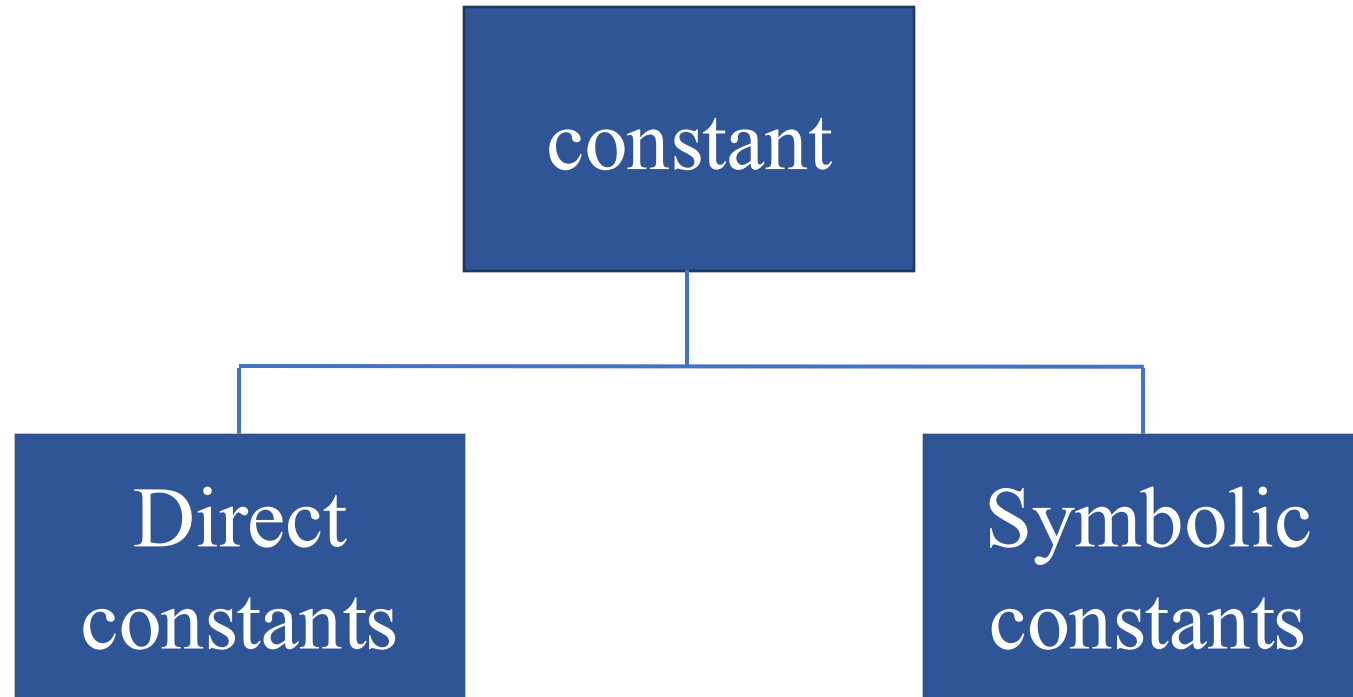
- ❑ The boolean type is used to determine the logical conditions, and is generally used for program flow control:
 - If Conditional control statements; while Loop control statements; do-while Loop control statements; for Loop control statements.
 - **Boolean can only be true or false**
 - You can't use 0 or non-0 integers instead of false and true, unlike in C.

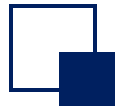
```
boolean b1 = true  
boolean b2 = false
```

Constants and variables

- **Constant:** A quantity whose value remains constant during the running of a program.





Constants and variables

- **Symbolic constants:** Constants represented by identifiers should be declared before they are used.

[modifier] final type identifier constant name = (direct) constant;

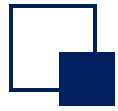
- A modifier is a permission modifier that indicates the scope of application of the constant, and the modifiers that can be adopted are:

public, private, protected or default

- **final** is a keyword that defines a constant
- There are three types of constants (with different scopes): static, member, and local

```
public final static Color black = new Color(0, 0, 0);  
public final static Color BLACK = black;  
public final static double PI = 3.1415926535;
```

1. Initialization is required at the time of definition
2. Constants are generally uppercase characters

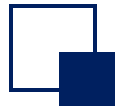


Constants and variables

- **Variable: refers to the amount of value that can be changed during the running of the program.**
Java stipulates that variables must be defined before they are used.

[Modifier] Type_Identifier Variable_Name [=Constant];

```
float x = 25.4, y;  
char c;  
boolean flag1 = true;
```



Conditional statements

□ If

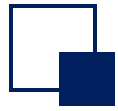
With the if conditional, you can choose whether you want to execute the statement immediately after the condition

```
int a = 100;
if(a == 100)
{
    System.out.println(a);
}
```

□ if...else

If a certain condition is met, one treatment is carried out, otherwise another treatment is carried out

```
if(math >= 60) { // 判断条件: math大于等于60成立
    System.out.println("math has passed");
} else { // 判断条件不满足
    System.out.println("math has not passed");
}
```



Conditional statements

□if...else if

If a certain condition is met, a certain treatment is carried out,

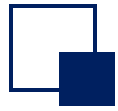
Otherwise, if another condition is met, another treatment is carried out.

□Switch Multi-branching statements

The value of the expression in the switch statement must be an integer, a character, or a string, and the constant value 1~n must also be an integer, character, or string.

```
if(x > 60) {  
    System.out.println("x is greater than 60");  
} else if (x > 30) {  
    System.out.println("x is greater than 30 but less than  
or equal to 60");  
} else if (x > 0) {  
    System.out.println("x is greater than 0 but less than  
or equal to 30");  
} else {  
    System.out.println("x is less than or equal to 0");  
}
```

```
switch (expression) {  
    case Constant value 1:  
        Statement block 1  
        [break;]  
  
    ...  
    case Constant value n:  
        Statement block n  
        [break;]  
  
    default:  
        Statement block n+1;  
        [break;]  
}
```



Loop statements

□while

The while loop statement is looped in such a way that a condition is used to control whether the statement should continue to be executed repeatedly.

```
while (x <= 10) {  
    sum += x;  
    x++;  
}
```

□do...while

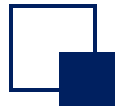
The difference between a **do...while** loop and a **while** loop statement is that the **while** loop statement first determines whether the condition is true and then executes the loop body, while the **do...while** loop is executed once before determining whether the condition is true. That is, **do...while** will be performed at least once.

```
do {  
    System.out.println("b == " + b);  
    b--;  
}while(b >= 8)
```

□for

A for loop can be used to repeat a statement until expression 2 is satisfied

```
for(Expression 1; Expression 2;  
Expression 3)  
{  
    Statement sequences  
}
```



Jump statement

The Java language provides three types of jump statements: break statement, continue statement, and return statement.

❑ break

The break statement can force exit the current loop. Suppose two loops are nested and break is used under the inner loop, then break can only jump out of the inner loop.

❑ continue

The continue statement is used to allow the program to skip the statement that follows it and proceed to the next loop.

❑ return

A return statement can return from a method and give control to the statement that called it.

```
for(int i=0; i<n; i++) {    // Outer Loop
    for(int j=0; j<n; j++) {    // inner Loop
        break;
    }
}
```

```
int i = 0;
while (i < 10) {
    i++;
    if(i % 2 == 0) {
        continue;    // If it's an even number,
        skip the current loop
    }
    System.out.println(i);
}
```

```
public void getName() {
    return name;
}
```



Classroom exercises

Please give the result of 4 pieces of code

```
int i = 0;
while (i < 10) {
    i++;
    if(i % 2 == 0) {
        continue; // If it's an even number,
        skip the current loop
    }
    System.out.println(i);
}
```

Results: 1, 3, 5, 7, 9

```
int i = 0;
while (i < 10) {
    if(i % 2 == 0) {
        continue; // If it's an even number,
        skip the current loop
    }
    System.out.println(i);
    i++;
}
```

Result: No output

```
int i = 0;
while (i < 10) {
    System.out.println(i);
    i++;
    if(i % 2 == 0) {
        continue; // If it's an even number,
        skip the current loop
    }
}
```

Results: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

```
int i = 0;
while (i < 10) {
    i++;
    if(i % 2 == 0) {
        break; // If it's an even number
        skip loop
    }
    System.out.println(i);
}
```

Results:1



Java arrays

❑ Declare an array

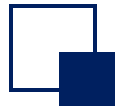
There are two ways to declare arrays, the first is the way the Java language specification promotes, and the second is the C-style way of declaring arrays.

```
int[] a;  
int b[];
```

❑ Create an array

There are two ways to create an array, the first one specifies the length of the array, and the second directly assigns a value to the array. Once an array is created, its length can no longer be changed.

```
int[] c = new int[2];  
int[] d = new int[] {0, 1};
```



Java arrays

Access the array

The subscript of the array starts from 0 and goes all the way up to the length -1. An array is a randomly accessed data structure.

```
int[] a = new int[2];  
for(int i = 0; i < 2; i++){  
    a[i] = i;  
    System.out.println(a[i]);  
}
```

Array length

The length property is used to access the length of an array

```
int[] a = new int[2];  
System.out.println(a.length);
```

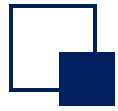
The array is out of bounds

The array access subscript range is 0 to length -1, and once this range is exceeded, an array subscript out-of-bounds exception will be generated

```
int[] a = new int[2];  
System.out.println(a[2]);
```



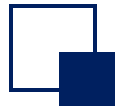
Java.Lang.ArrayIndexOutOfBoundsException: 2



Java exception keyword

❑ Exception keywords:

- **try** – Used for listening. Put the code to be listened to (code that may throw an exception) in the try block, and when an exception occurs in the try block, the exception will be thrown.
- **catch** – Used to catch exceptions. Catch is used to catch exceptions that occur in the try block.
- **finally** – Finally blocks will always be executed. It is mainly used to reclaim resources (such as database connections, network connections, and disk files) that are opened in the try block. Only after the final block is executed, the return or throw statement in the try or catch block will be executed. If a statement with a termination method such as return or throw is used in finally, the execution will not be jumped back and will be stopped.
- **throws** – Used in method signatures to declare exceptions that may be thrown by the method.
- **throw** – Used to throw exceptions.



Throw and throws

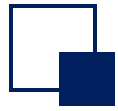
- **Common:**

Both are passive ways of handling exceptions, only responsible for throwing exceptions, but not handled by the function, the real handling of exceptions is handled by the upper layer of the function.

- **distinguish:**

- throws are used in the method header and represent only the declaration of the exception, while throw is used inside the method and throws the exception object.
- throws can throw multiple exceptions at once, while throw can only be one.

```
public static int getElement(int[] arr, int index) throws Exception, IOException {  
    if(arr==null) {  
        throw new NullPointerException("The pointer is empty");  
    }else if(index>=arr.length-1) {  
        throw new ArrayIndexOutOfBoundsException("The array is out of bounds");  
    }  
    int ele = arr[index];  
    return ele;  
}
```



Java exception mechanism

❑ Exceptions are usually caught and handled in the following ways:

- try-catch

Multiple exception types can be caught in a single try-catch block, and different types of exceptions can be handled differently

- try-finally

try block, the statement after the abnormal code is no longer executed, and the final statement is executed. If the try block does not throw an exception, the FINALLY statement is executed after the try block is executed.

- try-catch-finally

```
try{  
    // Execute the program code, an exception  
    may occur  
} catch(Exception1 e1) {  
    // Exception1 is caught and handled  
} catch(Exception2 e2) {  
    //Exception1 is caught and handled  
}  
...
```

```
try{  
    // Execute the program code, an exception  
    may occur  
} finally {  
    // Code that will definitely be executed  
...  
}
```

```
try{  
    // Execute the program code, an exception  
    may occur  
} catch(Exception e) {  
    // Exceptions are caught and handled  
} finally {  
    // Code that will definitely be executed  
}  
...
```